The chapter starts by refactoring the sample code from chapter 5 to be more configurable. It then takes a closer look at each of the CORS response headers and offers practical guidance on how to use them. By the end of this chapter, the sample code will become something you can incorporate into your own server to handle CORS requests.

6.1 Refactoring the sample code

The sample code from the previous three chapters did a good job of covering various CORS topics. But it falls far short of being a general-purpose CORS library. The code has hard-coded values geared toward explaining particular concepts, and isn't flexible or configurable. Concepts like the Timezone-Offset and Sample-Source request headers were made up to demonstrate CORS. If you were to copy and paste the sample code into your own server, it wouldn't match up with your own needs.

The goal of this chapter is to build a flexible CORS server that can be used with a variety of configurations. To do this, we'll first refactor the sample code so that it can be configured in different ways. The configuration options will allow the code to be used in various circumstances. The following listing shows the result of this refactoring.

Listing 6.1 Modifying the `handleCors` method to accept configuration options

Declares new configuration options object (to be used later in chapter)

```
var corsOptions = {};

var handleCors = function(options) {
  return function(req, res, next) {
    res.set('Access-Control-Allow-Origin', 'http://localhost:1111');
    res.set('Access-Control-Allow-Credentials', 'true');
    if (isPreflight(req)) {
      res.set('Access-Control-Allow-Methods', 'GET', 'DELETE');
      res.set('Access-Control-Allow-Headers',
              'Timezone-Offset, Sample-Source');
      res.set('Access-Control-Max-Age', '120');
      res.status(204).end();
      return;
    } else {
      res.set('Access-Control-Expose-Headers', 'X-Powered-By');
    }
    next();
  }
};

var SERVER_PORT = 9999;
var serverapp = express();
serverapp.use(cookieParser());
serverapp.use(express.static (__dirname));
serverapp.use(handleCors(corsOptions));
serverapp.get('/api/posts', function(req, res) {
  res.json(POSTS);
});
```

handleCors returns a new middleware function based off configuration options.

Adds new middleware to server.

The listing starts by declaring the `corsOptions` variable which will store the configuration options we'll add throughout this chapter. This variable will grow as you learn